PHONEBOOK MANAGEMENT SYSTEM

Abstract

This report details the implementation of a Phonebook Management System using a hash table to efficiently manage contacts. The system allows users to add, edit, delete, and search contacts by name, phone number, or email. It also supports importing contacts from CSV files, saving to a text file, and sorting by name or phone number. With fast access time (O(1) complexity) provided by the hash table and error checking for data integrity, the system offers a robust solution for managing contact information.

Problem Description

A phonebook application serves as a basic yet essential tool for storing, organizing, and retrieving contact details such as names, phone numbers, and emails. However, a simple phonebook application can become cumbersome when managing a large number of contacts, especially when performing operations such as adding, searching, updating, or deleting contacts.

The challenge addressed by this system is to create an efficient, user-friendly solution for managing a phonebook with a focus on:

- Scalability: The system should handle a growing number of contacts without significant performance degradation.
- Efficiency: Operations like searching, adding, editing, and deleting contacts must be fast and effective, even with a large dataset.
- Data Integrity: Ensuring that contact information is correctly stored and retrieved while preventing issues like data corruption or errors during input.
- User Experience: The system should be intuitive and allow the user to perform essential operations seamlessly.

Implementation Details

The Phonebook Management System is implemented in C, utilizing a hash table as the primary data structure. The system employs the following features:

- **Hash Table**: A hash table of fixed size is used to store contacts, allowing efficient insertion, deletion, and searching by name. Each bucket in the hash table points to a linked list of contacts.
- **Linked List**: Each hash table bucket contains a linked list, ensuring efficient handling of hash collisions.
- **File Handling**: Contacts are stored in a file (contacts.txt), and the system can import contacts from a CSV file and export them back to the text file for persistence.
- **Sorting**: The contacts can be sorted by name or phone number using the qsort function.
- **Validation**: Phone numbers are validated to ensure they are 10 digits long, and input errors are handled gracefully.

The menu-driven interface allows users to perform operations like adding, deleting, editing, and searching contacts, along with importing/exporting contacts and sorting them.

Data Structures Used

1. **Hash Table**: The primary data structure for storing contacts. The hash table's size is set to 100 to minimize collisions, and each bucket holds a linked list of contacts with the same hash value.
2. **Linked List**: Each contact is represented by a node containing fields for the name, phone number, and email. Linked lists handle potential hash collisions by chaining multiple contacts at the same index.
3. **Arrays**: Arrays are used to temporarily hold contacts for sorting operations.

Challenges Faced

- **Hash Collisions**: Efficient collision handling was a key challenge, which was addressed by using linked lists in each hash table bucket to manage multiple contacts at the same index.
- **Sorting**: Sorting contacts by name and phone number required custom comparison functions and careful memory management to ensure proper ordering.
- **Saving Contacts**: Initially, contacts weren't saved correctly upon program termination, leading to data loss. I fixed this by ensuring contacts were saved to a file and persisted after exiting the program.
- **Phone Number Validation**: The program didn't enforce the 10-digit phone number rule. I added validation to ensure only 10-digit numbers could be saved.
- **CSV Import**: The import process initially failed with malformed or incomplete data. I added error handling to skip invalid entries and ensure only valid contacts were imported.
- **Memory Management**: Early versions didn't free memory properly when contacts were deleted, causing memory leaks. I added code to free memory, preventing memory usage issues.

Conclusion and Learnings

The project successfully implemented a phonebook management system using a hash table for efficient contact storage and retrieval. Through this, I gained experience with hash tables, collision handling, and file operations in C. I also learned the importance of validating user input and handling errors effectively. A key takeaway was learning how to import and manage CSV files, ensuring proper data formatting and integration. Additionally, I improved my understanding of dynamic memory management to prevent leaks and optimized sorting techniques for handling large datasets. Overall, this project enhanced my C programming skills, deepened my understanding of data structures, and emphasized the importance of memory management, error handling, and file operations in building robust applications.

Screenshots:

1st time execution of code:

```
C:\Users\RUCHITA\Desktop\D

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice:
```

C:\Users\RUCHITA\Desktop\D

```
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 12
All contacts cleared successfully.

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 5
No contacts found.

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 6
3 contacts imported from CSV.

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 3
Enter name to delete: Kavya
Person deleted.
```

C:\Users\RUCHITA\Desktop\D

```
Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 1
Enter name: Ramu
Enter phone (10 digits): 1274928340
Enter email: ramu@gmail.com
Person added.

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 1
Enter name: Kavya
Enter phone (10 digits): 8897654230
Enter email: kavya@gmail.com
Person added.

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 5
Name: HariChandana, Phone: 9876543210, Email: hari@gmail.com
Name: Pravallika, Phone: 1234567890, Email: pravs@gmail.com
Name: Kavya, Phone: 8897654230, Email: kavya@gmail.com
Name: Ramu, Phone: 1274928340, Email: ramu@gmail.com

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 2
Enter name to edit: HariChandana
Enter phone (10 digits): 7723891230
Enter new email: hari@gmail.com
Person updated.
```

C:\Users\RUCHITA\Desktop\D

```
Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 7
Enter phone (10 digits): 8901273892
Not found.

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 8
Enter email: hari@gmail.com
Found: HariChandana,7723891230,hari@gmail.com

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 9
Contacts sorted by name:
Name: HariChandana, Phone: 7723891230, Email: hari@gmail.com
Name: Kavya, Phone: 8897654230, Email: kavya@gmail.com
Name: Pravallika, Phone: 1234567890, Email: pravs@gmail.com
Name: Ramu, Phone: 1274928340, Email: ramu@gmail.com

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 10
Contacts sorted by phone:
Name: Pravallika, Phone: 1234567890, Email: pravs@gmail.com
Name: Ramu, Phone: 1274928340, Email: ramu@gmail.com
Name: HariChandana, Phone: 7723891230, Email: hari@gmail.com
Name: Kavya, Phone: 8897654230, Email: kavya@gmail.com
```

```
C:\Users\RUCHITA\Desktop\D

10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 10
Contacts sorted by phone:
Name: Pravallika, Phone: 1234567890, Email: pravs@gmail.com
Name: Ramu, Phone: 1274928340, Email: ramu@gmail.com
Name: HariChandana, Phone: 7723891230, Email: hari@gmail.com
Name: Kavya, Phone: 8897654230, Email: kavya@gmail.com

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 11
Contacts saved to file.
Exiting.

--------------------------------
Process exited after 157.6 seconds with return value 0
Press any key to continue . . .
```

2nd time execution of code:

C:\Users\RUCHITA\Desktop\D

```
Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice: 5
Name: HariChandana, Phone: 7723891230, Email: hari@gmail.com
Name: Pravallika, Phone: 1234567890, Email: pravs@gmail.com
Name: Kavya, Phone: 8897654230, Email: kavya@gmail.com
Name: Ramu, Phone: 1274928340, Email: ramu@gmail.com

Phonebook Menu:
1. Add contact
2. Edit contact
3. Delete contact
4. Search by name
5. Display all contacts
6. Import contacts from CSV
7. Search by phone number
8. Search by email address
9. Sort contacts by name
10. Sort contacts by phone number
11. Save & exit
12. Clear all contacts
Enter your choice:
```